

Pre-Lecture Assignment 2 — Answers

Q1 — Non-uniform Polyak-Łojasiewicz (PL)

(a) Why "non-uniform"?

In the standard (uniform) PL condition, the gradient lower bounds the optimality gap with a **constant** coefficient independent of θ . Here, the coefficient

$$C(\theta) = \frac{\min_s \pi_\theta(a^*(s) | s)}{\sqrt{S} \cdot \|d_\rho^{\pi^*} / d_\mu^{\pi_\theta}\|_\infty}$$

depends on θ : both $\min_s \pi_\theta(a^*(s) | s)$ and $d_\mu^{\pi_\theta}$ change as the policy updates. The PL "constant" is non-uniform across parameter space — large when π_θ is near-optimal, vanishingly small otherwise.

(b) Origin of $\min_s \pi_\theta(a^*(s) | s)$

The proof of Lemma 8 proceeds in three key steps:

Step 1 — Restrict to optimal-action coordinates (Cauchy-Schwarz):

$$\|\nabla_\theta V^{\pi_\theta}(\mu)\|_2 \geq \frac{1}{\sqrt{S}} \sum_s \left| \frac{\partial V^{\pi_\theta}(\mu)}{\partial \theta_{s, a^*(s)}} \right|$$

Step 2 — Substitute softmax PG formula (Lemma 1): $\frac{\partial V^{\pi_\theta}(\mu)}{\partial \theta_{s, a}} = \frac{1}{1-\gamma} d_\mu^{\pi_\theta}(s) \pi_\theta(a|s) A^{\pi_\theta}(s, a)$. At $a = a^*(s)$, the factor $\pi_\theta(a^*(s)|s)$

appears inside the sum — it comes from the policy probability in the softmax gradient.

Step 3 — Extract min to apply the performance difference lemma:

$$\sum_s d_\mu^{\pi_\theta}(s) \pi_\theta(a^*(s)|s) A^{\pi_\theta}(s, a^*(s)) \geq \min_s \pi_\theta(a^*(s)|s) \cdot \sum_s d_\mu^{\pi_\theta}(s) A^{\pi_\theta}(s, a^*(s))$$

Pulling out the min makes the remaining sum convertible to $V^*(\rho) - V^{\pi_\theta}(\rho)$ via the performance difference lemma. In summary:

$\min_s \pi_\theta(a^*(s)|s)$ originates from the **softmax gradient formula** (Step 2) and is **extracted via a min lower-bound** (Step 3) to enable the final conversion to the optimality gap.

(c) Difference between μ and ρ

- μ : initial state distribution for **gradient computation** — determines $d_\mu^{\pi_\theta}$ used in the policy gradient update.
- ρ : initial state distribution for **performance evaluation** — the optimality gap $V^*(\rho) - V^{\pi_\theta}(\rho)$ is measured under ρ .

They can differ. The ratio $\|d_\rho^{\pi^*} / d_\mu^{\pi_\theta}\|_\infty$ captures the distribution mismatch between training (μ) and evaluation (ρ) — the larger it is, the harder convergence becomes.

Q2 — QAC Algorithm (Lines 100–119)

(a) Math Equations

Critic forward (L101–102): $Q_w(s_t, a_t) = [\text{critic}(s_t)]_{a_t}$

TD target (L104–112) — Expected SARSA:

$$\hat{V}^{\pi_\theta}(s_{t+1}) = \sum_a \pi_\theta(a | s_{t+1}) Q_w(s_{t+1}, a), \quad y_t = \begin{cases} r_t & \text{if done} \\ r_t + \gamma \hat{V}^{\pi_\theta}(s_{t+1}) & \text{otherwise} \end{cases}$$

Rather than bootstrapping with a single sampled next action, the target uses the policy-weighted expectation $\mathbb{E}_{a \sim \pi_\theta}[Q_w(s_{t+1}, a)]$, which equals $V^{\pi_\theta}(s_{t+1})$ under the current critic.

Critic update (L115–119): $\mathcal{L}(w) = (Q_w(s_t, a_t) - y_t)^2$, $w \leftarrow w - \alpha_w \nabla_w \mathcal{L}(w)$

(b) Do you agree with GPT? — No, two issues.

Issue 1 (algorithmic): Standard QAC uses a SARSA-style target $y_t = r_t + \gamma Q_w(s_{t+1}, a_{t+1})$ where $a_{t+1} \sim \pi_\theta$. GPT uses Expected SARSA instead, which deviates from the standard QAC formulation (though it reduces variance).

Issue 2 (terminal handling): Line 112 uses `done = terminated or truncated` (L94) to stop bootstrapping. When an episode is **truncated** (max steps reached), it is not truly terminal — bootstrapping should still occur. The condition should use only **terminated**:

$$y_t = r_t + \gamma (1 - \mathbb{1}[\text{terminated}]) \hat{V}^{\pi_\theta}(s_{t+1})$$